Beyond 297 Passing Tests: Auditing Exception Contracts in Local Gemma Repairs

An executable case study and six diagnostic tasks with calibrated validation gates

Jacek Adamek | Poland

ABSTRACT

A generated repair can pass a complete upstream test suite while changing an exception contract. We document such a case for local Gemma 4 E2B: an ItsDangerous timestamp repair passes 297 tests but replaces the original bytes payload with a diagnostic string. Neither of two feedback-guided alternatives resolves the original failure. We introduce Exception Contract Audit (ECA-6), a small original diagnostic resource containing six related repair tasks, reference implementations, weak and contract-focused tests, and 36 deliberately flawed calibration patches. All tasks and initial evaluation rules are frozen before six new one-shot model generations. All six outputs pass the initial tests. A separately labeled post-outcome audit detects an existing-message regression in one output; five pass the expanded checks. The expanded gate also rejects all 36 calibration patches, versus none rejected by the weak gate. These are bounded diagnostic findings, not an estimated real-world error rate, a novel differential-testing algorithm, or proof of patch correctness. The contribution is an auditable resource and a concrete local-model counterexample, including unsuccessful attempts and limitations of the validators themselves.

1. INTRODUCTION

Local coding assistants need affordable validation. A developer may reasonably treat a patch passing an entire project suite as substantially stronger evidence than a patch passing only a selected test file. Our motivating observation shows a remaining gap: a library exception can have the expected type while carrying changed metadata that existing assertions do not inspect.

We ask three questions. RQ1: Can an actual local Gemma repair pass all upstream tests while violating a concrete exception property? RQ2: Can a compact executable resource distinguish different exception-contract regressions, and expose weaknesses in its own checks? RQ3: What happens when the same local model receives fresh, explicitly contract-specified diagnostic tasks?

The contribution is ECA-6 and its reproducible evaluation record, together with a precisely scoped ItsDangerous counterexample. It includes frozen prompts, candidate and reference source hashes, complete outputs, failed attempts, exact test logs, calibration patches and separately versioned audit gates. We do not invent regression testing, mutation testing, differential testing or exception contracts. Originality is limited to this resource, measurements and case analysis; no priority over all related resources is asserted.

2. RELATED WORK

SWE-bench evaluates repository-level issue resolution through executable tests [1]. SWE-Bench+ reports weaknesses in benchmark tests and possible solution leakage [2]. Wang, Pradel and Liu's PatchDiff study distinguishes plausible test-passing patches from behaviorally divergent patches using differential testing and manual assessment [3]. Our case is consistent with that established concern. We do not reimplement or outperform PatchDiff: our checks are manually specified, narrowly target exception fields, and use known reference behavior.

Rothermel and Harrold study safe regression-test selection under explicit assumptions [4]. Our earlier ImportGuard experiments use a coarse static import graph without such a safety proof. They motivate this investigation but are not presented as a new safe selector. A frozen upstream study selected entire declared pools, yielding no module saving. The present paper therefore focuses on validation quality rather than claiming a speedup from selection.

3. METHODS AND PROVENANCE

3.1 Local inference

All reported generations use Google's Gemma 4 E2B QAT Q4_0 GGUF, revision 675cff42a74c774d6cb76f76d8eacb49b48c9b93 [5], with verified weight SHA-256 recorded in the artifact. The runtime is llama.cpp b11429, commit d81235049 [6], with an executable hash. Generation runs locally on CPU with two threads, temperature zero, seed 20261015, 4096-token context, reasoning disabled and no model tools. The ECA-6 budget is 512 generated tokens per task, one attempt each, without feedback or retries. Runtime and raw text are recorded; no training or paid inference is used.

These are source-edit suggestions, not autonomous repository-solving agents. Codex generated scaffolding and tests, reviewed proposed code for local execution, conducted analyses and assisted writing. Generated edits were applied exactly after review; no patch was manually repaired and then credited to Gemma. Original ECA-6 code is Apache-2.0. Third-party snippets retain their upstream licenses. Neither model weights nor complete upstream repositories are redistributed.

3.2 ItsDangerous case

The public Windows timestamp correction is commit 177196d3a5c7ad37b8ccbd0f88445aced07f1ca0 [7]. Fixed-version tests are held constant while the changed production file is replaced by parent source. Archive hashes, source/test hashes and the BSD license are retained. The initial attempt lacked freezegun and failed collection; that outcome remains recorded. A separate dependency follow-up uses freezegun 1.5.5 and pytest 8.4.2.

The fixed timed module passes 101 tests; parent source fails test_malformed_future_timestamp with OSError and passes 100. The complete fixed suite passes 297 tests; parent source fails one and passes 296. The model sees buggy functions and a symptom description, not the fixed patch or test source. Localization is manual, and this public case may occur in pretraining; no contamination-free claim is possible.

The first 256-token response is truncated and never applied. An identical-prompt attempt with a 1024-token budget produces a complete edit. After testing and differential inspection, two further 512-token attempts receive progressively stronger feedback. These are four distinct attempts, not four independent tasks or a pass@k capability estimate.

3.3 ECA-6 and calibration

ECA-6 contains six newly authored, templated Python tasks. Each wraps a low-level range error in ContractError while preserving a payload object, stamp=None and exception chaining. Tasks vary the additional exception family and nominated payload domain: OSError/bytes, OverflowError/mapping, KeyError/text, IndexError/bytearray, TypeError/tuple and ZeroDivisionError/list. Valid inputs and an unrelated RuntimeError supply controls. The reference extends the existing wrapper handler while leaving the low-level helper unchanged.

Before new model outcomes, we freeze all six sources, prompts, references, tests and calibration mutations. Prompts explicitly request contract preservation but withhold test, reference and mutant code. The initial weak gate contains three tests: one valid input and negative/range inputs that check the wrapper exception type. Contract gate v1 adds four tests covering payload identity, stamp, cause type, direct-helper behavior, unrelated-error propagation and additional valid inputs.

Each reference also yields six deliberately flawed calibration variants: stringified payload, removed cause, changed stamp, overly broad catch, changed helper exception, and a valid-input regression outside the weak example. These 36 constructed variants test the sensitivity of a validator; they are not model errors sampled from use in the wild. Reference implementations must pass both gates; buggy implementations must fail the weak gate. All outcomes are retained.

After inspecting the first results, we separately freeze v2, adding checks for the existing negative-input error message and identity across additional payload domains. We evaluate the same reviewed outputs and calibration variants without rerunning the model. V2 is explicitly post-outcome development, not a prospective or independent validation result.

4. RESULTS

4.1 A complete-suite success with changed metadata

The 1024-token ItsDangerous proposal catches OSError in timestamp_to_datetime and raises BadTimeSignature there. It passes all 101 timed-module tests and all 297 upstream tests. However, the malformed input produces a string payload containing a timestamp and diagnostic error rather than the reference bytes b'value'. The upstream test checks exception type, message content and date_signed, but not payload. A full suite therefore does not certify this metadata property in this case.

The first feedback-informed proposal and the further localized proposal change the earlier integer-decoding handler, leaving the failing datetime conversion unguarded. Each retains one upstream failure. Three additional post-outcome checks cover original payload, valid roundtrip and direct-conversion exception behavior: the reference passes all three, while the latest proposal fails the payload-path check. A manually reconstructed upstream-equivalent correction passes 297 tests plus these checks, but is explicitly not counted as a model repair or original bug discovery.

4.2 Prospective diagnostic results and post-outcome audit

Group	Count	Weak	V1	V2
Reference	6	6	6	6
Buggy	6	0	--	--
Calibration	36	36	1	0
Gemma	6	6	6	5

All six model responses are complete, uniquely applicable JSON edits; none is retried or manually corrected. They pass all initial checks. V1 rejects 35 of 36 calibration variants. The remaining stringification variant is indistinguishable on its nominated text payload because str(text) preserves that object in the tested case; no equivalence for arbitrary payloads is claimed. V2's additional payload domains expose it, rejecting all 36 deliberately flawed variants while retaining all references.

V2 also detects an existing-error-message regression in the TypeError/tuple model output: the original negative-input message 'bad' becomes 'out-of-range'. Both the weak gate and v1 miss it. The other five outputs pass v2. This is a measured difference between declared gates on these artifacts, not proof that v2 is complete. Several outputs additionally introduce branching or new range-error messages; unspecified new-case message differences are not automatically classified as faults.

5. INTERPRETATION AND LIMITATIONS

The useful observation is not that Gemma always fails. It succeeds on the initial tests for six simple, explicitly specified tasks, yet a real-library patch and one diagnostic output reveal properties those tests omit. Calibration makes the validator's scope inspectable: v1's single-payload check misses one deliberately flawed variant, and v2 removes that observed blind spot. Test metadata should therefore be reported as checked properties, not an unqualified semantic-correctness label.

Six templated tasks are strongly correlated and intentionally easy to localize. They were designed after the ItsDangerous observation, by the same assisted development process that evaluates them. They do not constitute independent task authorship, a random repository sample or an estimate of Gemma's general error rate. Calibration mutants encode anticipated faults, not their natural prevalence. V2 is informed by outcomes, so its 5/6 acceptance and 36/36 rejection are developmental measurements. There is no fresh held-out validation of v2.

We use one quantized small model, one runtime, one seed and CPU inference. Results do not transfer automatically to Gemma 31B, other quantizations, platforms or autonomous agents. The real-library behavior is Windows-specific; the constructed tasks explicitly raise exceptions and are platform-portable. Error payload and message compatibility matter in the specified contracts, but reference differences alone are not a universal correctness oracle. No leaderboard result, selection speedup, causal benefit from feedback, safety proof or new general testing algorithm is claimed.

6. REPRODUCIBILITY AND CONCLUSION

The package contains contract_resource.py, its frozen JSON protocol, references, tests and all 36 calibration sources; text-only inference, reviewed-patch evaluation and the separately named posthoc-v2 audit; exact local-model outputs and per-run test logs. The resource can be evaluated using standard Python and pytest without downloading Gemma. A review notebook reproduces recorded summaries and explains optional inference with separately acquired pinned weights/runtime. Earlier ImportGuard and upstream studies remain available as historical supporting material, including unsuccessful and uninformative cases.

This study contributes a concrete Gemma exception-contract counterexample and an executable diagnostic resource that exposes both patch errors and validator limitations. Its value is an auditable distinction among upstream-test success, specified-contract success and unverified behavior. Larger independent studies are needed before treating the gates as general validation methods.

REFERENCES

[1] Carlos E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024. https://arxiv.org/abs/2310.06770

[2] Reem Aleithan et al. SWE-Bench+: Enhanced Coding Benchmark for LLMs. 2024. https://arxiv.org/abs/2410.06992

[3] You Wang, Michael Pradel and Zhongxin Liu. Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study. ICSE 2026. https://doi.org/10.1145/3744916.3764576 ; https://software-lab.org/publications/icse2026_SWE-bench-correctness.pdf

[4] Gregg Rothermel and Mary Jean Harrold. A Safe, Efficient Regression Test Selection Technique. ACM TOSEM 6(2), 1997. https://doi.org/10.1145/248233.248262

[5] Google DeepMind. Gemma 4 E2B QAT Q4_0 GGUF model. https://huggingface.co/google/gemma-4-E2B-it-qat-q4_0-gguf

[6] ggml-org and contributors. llama.cpp b11429. https://github.com/ggml-org/llama.cpp/releases/tag/b11429

[7] Pallets contributors. ItsDangerous Windows exception correction. https://github.com/pallets/itsdangerous/commit/177196d3a5c7ad37b8ccbd0f88445aced07f1ca0